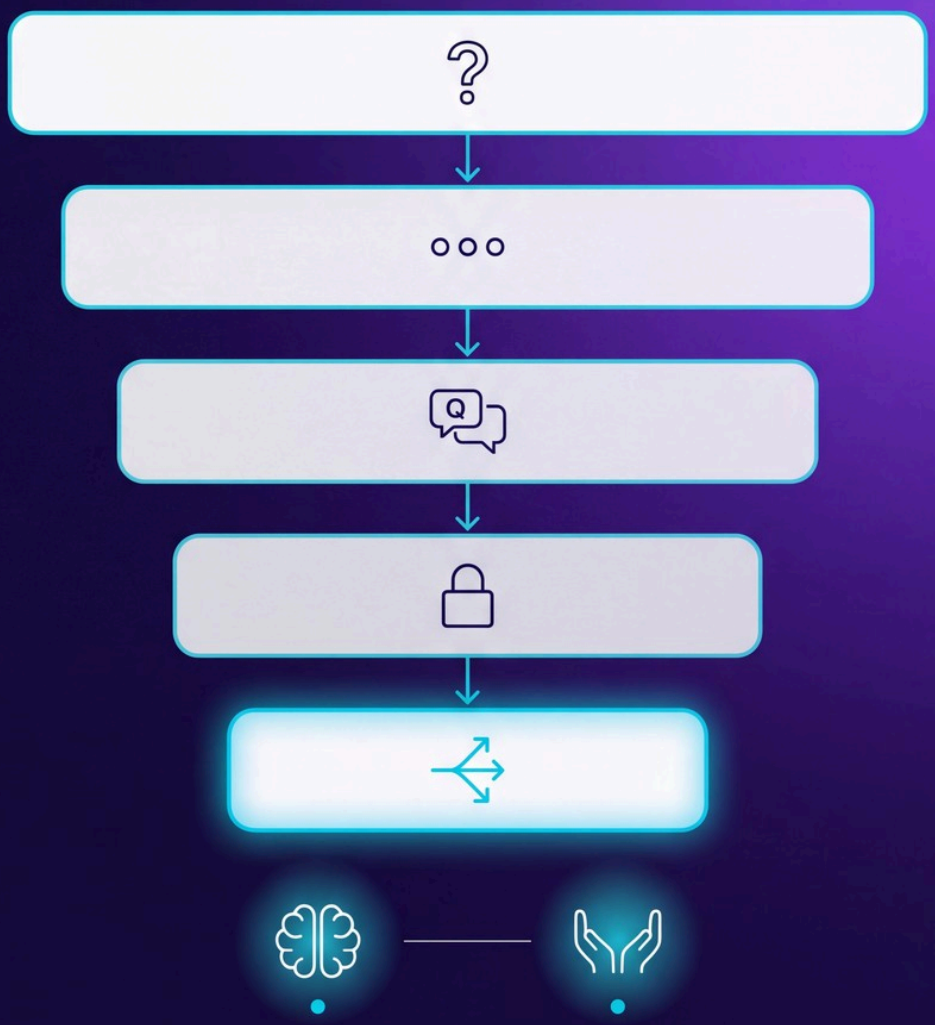
FREE PLAYBOOK

Agentic Architecture for Vibe Founders

The 5-layer questionnaire system. Dual model routing. **Parallel task fan-out.** The 90-day build plan.



AI BrandFactory

The 5-layer questionnaire system, dual-model routing (smart brain plus cheap hands), parallel task fan-out, the storage and resumability model, the toolkit, and a 90-day build plan. Builder-to-builder, slightly more technical than the marketing playbooks. Worked example throughout: a launch-kit generator for a new SaaS that fans out 200 assets in parallel.

Who this is for

Solo founders building agentic AI products in 2026. Indie hackers who ship in days, not quarters. Devs designing their first multi-agent system and trying to figure out which advice in their feed is real and which is content marketing for somebody else's framework.

Most agentic AI content is written by people who have never shipped an agent. They build for engineering teams that do not exist in your stack. Vibe founders need a different playbook. You ship in days, not quarters. You pick boring infrastructure that does not need a Kubernetes cluster. You route every task by cost-per-token before you route by anything else. And you treat the user as a co-author of the spec, not a downstream consumer of it.

I have shipped 4 agentic products as a solo founder in the last 18 months. The architecture in this playbook is the one I keep coming back to. Seven parts, builder-to-builder, no theory padding. The worked example throughout is a launch-kit generator for a new SaaS, where one founder runs a questionnaire once and the system fans out 200 tasks (cover image, landing copy, ads, emails, social posts, comparison page, FAQ) in parallel and ships them into the founder's account ready to ship.

> **Brought to you by AI BrandFactory.** AI BrandFactory is a free lead magnet library for founders, marketers, and operators. Playbooks, prompts, frameworks, and checklists at files.aibrandfactory.com. Browse the full set at aibrandfactory.com.

What is in this playbook

- **Part 1**, The Vibe Founder vs Enterprise Architect
- **Part 2**, The 5-Layer Questionnaire Architecture
- **Part 3**, Dual Model Routing: Smart Brain plus Cheap Hands

- **Part 4**, Parallel Task Fan-Out
- **Part 5**, State, Storage, and Resumability
- **Part 6**, The Vibe Founder's Toolkit
- **Part 7**, The 90-Day Build Plan

Part 1, The Vibe Founder vs Enterprise Architect

There are two kinds of people writing about agentic AI right now and almost everything you read is from the wrong one.

The enterprise architect writes for a 50-engineer team. They have a platform org. They have a security review board. They have a 6-month roadmap that gets quarterly steering committee approval. Their stack is 14 services deep and there is a dedicated team owning each layer. When they say "agentic system" they mean a multi-tenant SaaS platform that has to satisfy SOC 2 and EU residency rules on day one.

The vibe founder is one person on a laptop. Maybe two if you count the contractor in another timezone. You ship a working v1 by the end of the week or you do not eat next month. Your stack is whatever runs on Railway for $20 a month. Your security review is a quick scan of the OWASP top 10 because you are responsible and not because anyone told you to.

These are not the same problem. And the same architecture answer breaks both ways. The enterprise architect's pattern is over-engineered for your stack and you will burn 3 weeks setting up the orchestration layer before you ship a single agent. The vibe founder's pattern is under-engineered for theirs and the security team will block the deploy.

Most agentic AI content is written by and for the enterprise side. Useless for you. Here are the 5 design philosophies that flip when you are shipping solo.

Build small first

Pick the smallest possible scope that delivers user value end to end. If your product is a launch-kit generator, the v1 is one questionnaire layer that produces 5 assets, not 5 layers producing 200. You can always grow the scope. You cannot un-ship the 6 weeks you spent building the framework before you talked to a paying user.

Ship in days, not quarters

If your build cycle is longer than two weeks for any single capability, you have over-scoped it. Cut the scope, not the timeline. The vibe founder's superpower is the ability to ship today what an enterprise team would still be in design review for next month. Use it.

Pick boring infrastructure

Postgres on Supabase. Node.js or Python in one file. Railway or Fly for deploy. JSONL on disk for state until you outgrow it. S3 for blobs. That is the entire stack. You do not need Kubernetes. You do not need a message queue. You do not need a dedicated vector database when Postgres has pgvector and your embedding count is under 100k.

Route by cost, not by aesthetics

Every API call costs money. The vibe founder runs the math on every token. If a task can be done by a model that is 9x cheaper and 80 percent as good, the cheap model wins for every task that does not need judgement. We will dig into this in Part 3.

Treat the user as a co-author of the spec

The enterprise pattern assumes the spec is fixed before the agent runs. The vibe founder pattern assumes the spec gets refined inside the system itself, in conversation with the user. The questionnaire architecture in Part 2 is exactly this: the user does not write a perfect prompt, they answer 5 layers of questions and the system writes the prompt for them.

These five philosophies shape every decision in the rest of this playbook.

A quick check on whether this playbook is for you

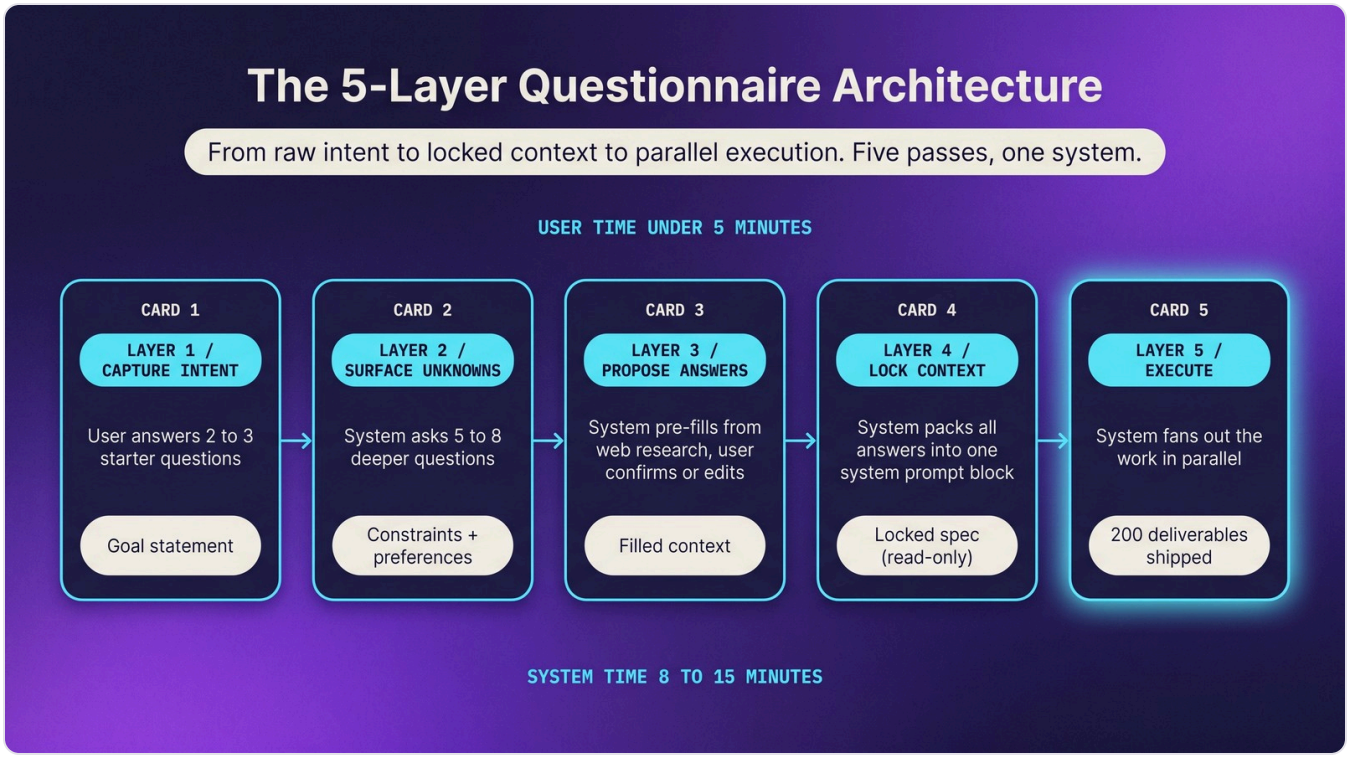
If you have an engineering team of 8 or more, a platform org, or a product manager whose full-time job is the agentic feature roadmap, this playbook is too small for you. Read the LangChain docs and the OpenAI Assistants reference instead.

If you are one to three people on a laptop, shipping in two-week cycles, and the AWS bill is something you actually look at line by line every month, you are in the right place. Keep reading.

Part 2, The 5-Layer Questionnaire Architecture

Most agentic systems fail at the front door. They assume the user can describe what they want in one paragraph. They cannot. The user knows their goal in vibes and the system is supposed to translate vibes to a working spec. That is the job.

The 5-layer questionnaire walks the user from raw intent to locked context across five progressively deeper passes. Each layer adds detail. Each layer is shown to the user. Each layer can be edited.



The shape:

Layer	What it does	Who fills it in	Output
Layer 1	Capture intent	User answers 2 to 3 questions	Goal statement
Layer 2	Surface unknowns	System asks 5 to 8 deeper questions	Constraints and preferences
Layer 3	Propose answers	System pre-fills from web research, user confirms or edits	Filled context
Layer 4	Lock context	System packs everything into one system prompt block	Locked spec
Layer 5	Execute	System fans out the work in parallel	Deliverables

Let me walk each one in detail. Worked example throughout: a SaaS founder named Maya is launching a new project management tool for design agencies. She wants a launch kit. She points the system at her draft landing page URL and clicks Start.

Layer 1, Capture intent

Two to three questions. Not five. Not eight. The user has not committed yet. You are still earning the right to ask deeper questions.

For Maya's launch kit, Layer 1 is:

1. What are you launching? (Free text, one line.)
2. Who is it for? (Free text, one line.)
3. What is the single most important thing you want to happen in the next 30 days? (Free text, one paragraph.)

That is it. Maya answers. The system has the goal.

Layer 2, Surface unknowns

Now the system asks what it needs to do good work. Not what is intellectually interesting. Not what would be nice to have. What it needs.

For a launch kit, Layer 2 might be:

- What is your primary acquisition channel for this launch? (Multi-select: Product Hunt, Twitter, LinkedIn, paid ads, partner outreach, none yet.)
- What is your target signup count for the first 30 days? (Number input, slider 50 to 5000.)
- What is your pricing model? (Single select: free, freemium, paid trial, paid only.)
- Who do you compete with that the user already knows? (Multi-select with text input fallback.)
- What are 3 things your product does that the competitor cannot do? (Free text, three lines.)

Five to eight questions. Each one feeds a downstream decision. If acquisition channel is Twitter, the system will generate Twitter threads. If it is Product Hunt, the system will generate the PH listing copy. The questions are not for show, they route the work.

Layer 3, Propose answers

This is the layer most agentic systems skip and it is the one that earns the user's trust. The system goes off, does web research on the things it can research, and proposes pre-filled answers for the user to confirm or edit.

For Maya's launch kit, Layer 3 might present:

- "Based on your draft landing page, here is what your product does in one paragraph: [generated paragraph]. Is this right? [Confirm / Edit]"
- "Here are the top 5 keywords your competitors rank for in your space: [list from a search API]. Which of these matter to you? [Multi-select with edit]"
- "Here are 8 angles your competitors are using in their launch copy: [list]. Which 3 do you want to avoid? [Multi-select]"
- "Here is your suggested tone of voice based on your existing copy: [3 adjectives]. Confirm or edit? [Confirm / Edit]"

The user is not generating from scratch. They are confirming or correcting a draft the system already filled in. This is way faster, way more accurate, and removes the blank-page problem.

Layer 4, Lock context

The system packs everything from Layers 1 to 3 into one consolidated system prompt block. Not split across 5 messages. Not stored in 12 different fields. One block. This becomes the system prompt for every downstream task.

For Maya the locked context is roughly 1,200 to 1,800 tokens. Goal, audience, channel, pricing, competitors, differentiators, keywords, tone, angles to avoid. That block is now read-only for the duration of the run. If Maya wants to change something, she goes back to Layer 3 and edits.

The lock matters. When you fan out 200 tasks in Layer 5 and 184 of them succeed, you do not want any of them to have used a slightly different version of the context. Locking the context is what makes the outputs feel coherent across all 200.

Layer 5, Execute

Now the system fans out the work. For Maya's launch kit, that is roughly 200 tasks: a hero image, a launch tweet thread, a Product Hunt listing draft, 8 ad creatives, 12 social posts spread across the launch week, 5 cold email templates for partner outreach, an FAQ page, a comparison page against the top 3 competitors, and so on.

Every one of those tasks runs against the same locked context. Every one routes through the dual-model router (Part 3). Every one fans out in parallel (Part 4). Every one is resumable if the system crashes (Part 5).

The user sees a progress bar and a stream of completed tasks. About 8 minutes later, Maya has a complete launch kit sitting in her dashboard.

How long does this take the user?

Layers 1 and 2 are fast. 30 seconds for Layer 1, 90 seconds for Layer 2. The user is filling in form fields they already know the answers to.

Layer 3 is the slowest because the user is reviewing the system's pre-filled work. Budget 2 to 4 minutes here. The pre-fill is what saves the time vs the user typing every answer fresh.

Layer 4 is invisible to the user, it is the system packing the locked context.

Layer 5 the user does not wait for synchronously. They close the tab. They get a notification when the deliverables are ready, somewhere between 8 and 15 minutes later depending on batch size.

Total user time inside the questionnaire: under 5 minutes. That is the budget. If your questionnaire takes longer, cut questions until it fits.

Why this architecture beats one-shot prompting

You could try to do this with one giant prompt. "Generate me a launch kit for my SaaS." The output would be generic, low-fidelity, and contradictory across the 200 assets because the model loses track of constraints across a long generation.

The 5-layer pattern beats one-shot prompting because:

- The user inputs are smaller and more specific at each step.
- The system pre-fills the boring parts so the user only has to confirm.
- The locked context is reused across every downstream task so the outputs feel coherent.
- The 5 layers themselves are fast (under 4 minutes total of user time) and the heavy lift happens after the user closes the tab.

This is the core idea. Everything else in the playbook is implementation detail.

Part 3, Dual Model Routing: Smart Brain plus Cheap Hands

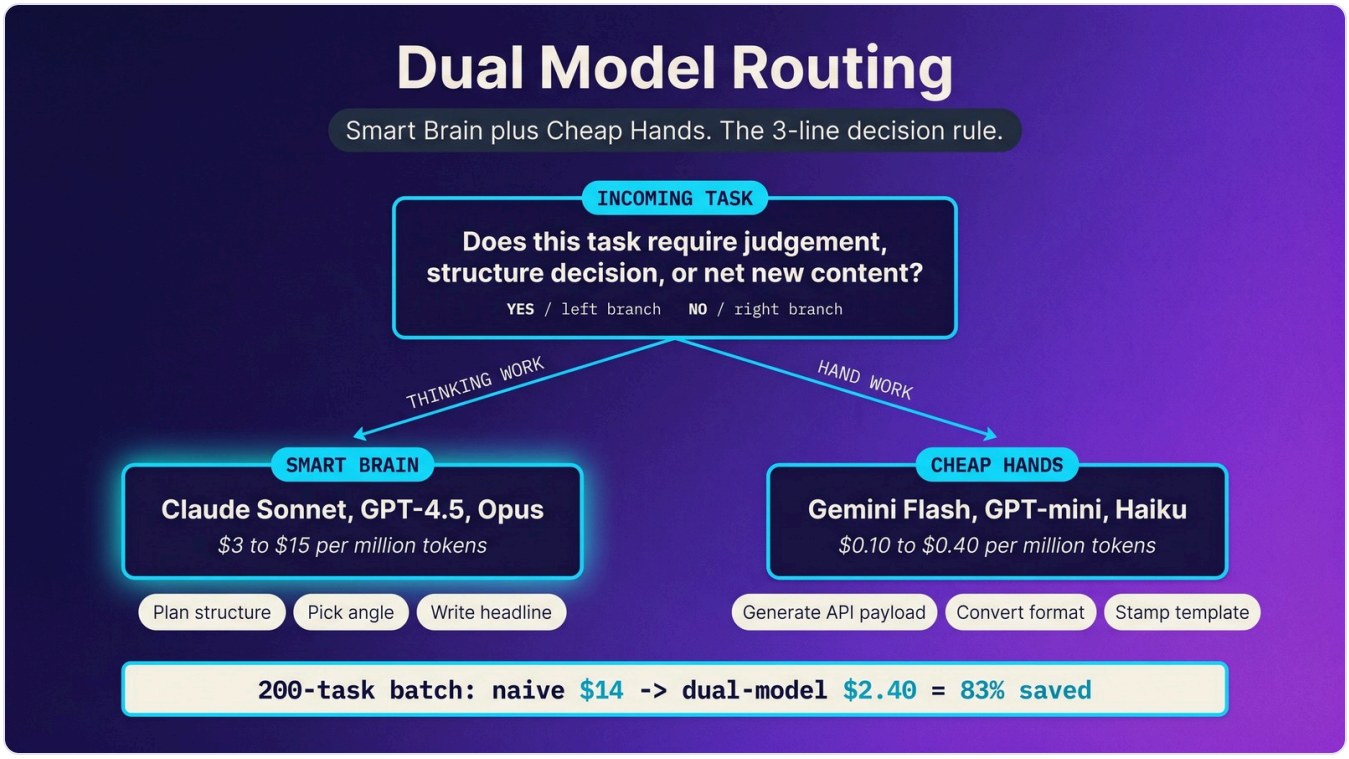
Here is the cost-quality split that pays the rent.

You have two kinds of work happening inside an agentic system:

Thinking work. Planning the structure of a deliverable. Deciding whether to use one approach or another. Writing the headline of the landing page. Choosing the angle for an ad. Judgement calls. These need a smart model.

Hand work. Generating the API payload for the next call. Converting a draft into JSON. Reformatting markdown into HTML. Stamping out a templated email with three variables filled in. Repetitive transformations that follow a clear shape. These do not need a smart model. They need a fast cheap one.

Most agentic systems route everything through the smart model because that is the path of least resistance during prototyping. Then the bill arrives and the founder learns this lesson the expensive way.



The 3-line decision rule

Paste this into your routing function. It works:

```
If the task requires judgement, structure decision, or net new content, use the smart
model.
If the task is a transformation, format conversion, or templated payload, use the cheap
model.
If you are not sure, default to the cheap model and check the output quality on a sample
of 10 runs.
```

That is it. No ML model picking the model for you. No complicated decision tree. A 3-line rule a junior developer can implement in an hour.

The model lineup as of 2026

The smart side:

- Claude Sonnet 4.6 for general thinking work. Best price-quality on judgement calls.
- Claude Opus 4 for the rare task that needs deeper reasoning. 5x the cost of Sonnet, use sparingly.
- GPT-4.5 for tasks where you specifically want OpenAI's tuning style.

The cheap side:

- Claude Haiku 4 for the routing decision itself and any cheap task in the Anthropic family.
- Gemini 2.5 Flash for high-volume payload generation. Roughly $0.10 per million input tokens, $0.40 per million output. Cheap enough that you stop counting.
- GPT-4-mini for OpenAI ecosystem tasks where you need API parity with the smart side.

The lineup will shift quarterly. The principle does not. Use the smart model for thinking, the cheap model for hands.

Worked cost math, the 200-task batch

Maya's launch kit is 200 tasks. Let me run the cost math both ways.

Naive routing: every task goes through Claude Sonnet. Average input tokens per task: 2,400. Average output: 1,200. At Sonnet pricing roughly $3 per million input and $15 per million output:

- 200 tasks x 2,400 input = 480,000 input tokens x $3 / 1M = $1.44
- 200 tasks x 1,200 output = 240,000 output tokens x $15 / 1M = $3.60
- Total: $5.04 per launch kit

Now multiply by image generation, embeddings, and the structured payloads each task needs to land in Maya's account. Real total for the naive route: about $14 per launch kit.

Dual-model routing: the 30 percent of tasks that are judgement calls (writing the landing copy, picking the angle, drafting the cold emails) stay on Sonnet. The 70 percent that are payload generation (formatting the social posts to fit each platform's API, generating the ad creative payloads, stamping out the FAQ page from a template) drop to Gemini Flash.

- 60 thinking tasks at Sonnet pricing: 60 x ($0.0072 + $0.018) = $1.51
- 140 hand tasks at Gemini Flash pricing: 140 x ($0.00024 + $0.00048) = $0.10
- Total: $1.61 per launch kit, plus the same image and structured-payload overhead from before.

Real total for the dual-model route: about $2.40 per launch kit.

That is 83 percent saved on the same workload. At 1,000 launch kits a month, the naive route costs $14,000. The dual-model route costs $2,400. The difference funds another contractor or three months of marketing.

What goes wrong

The two failure modes:

The first is sending judgement work to the cheap model. Gemini Flash will happily write you a landing page headline. It will not be a good headline. The cheap models do not do voice. They do shape. Keep voice work on the smart side.

The second is over-engineering the router. Some teams build a meta-model that picks the model for each task. Do not do this. The 3-line decision rule is enough. Save the engineering for the actual product.

A note on Claude Haiku as the router itself

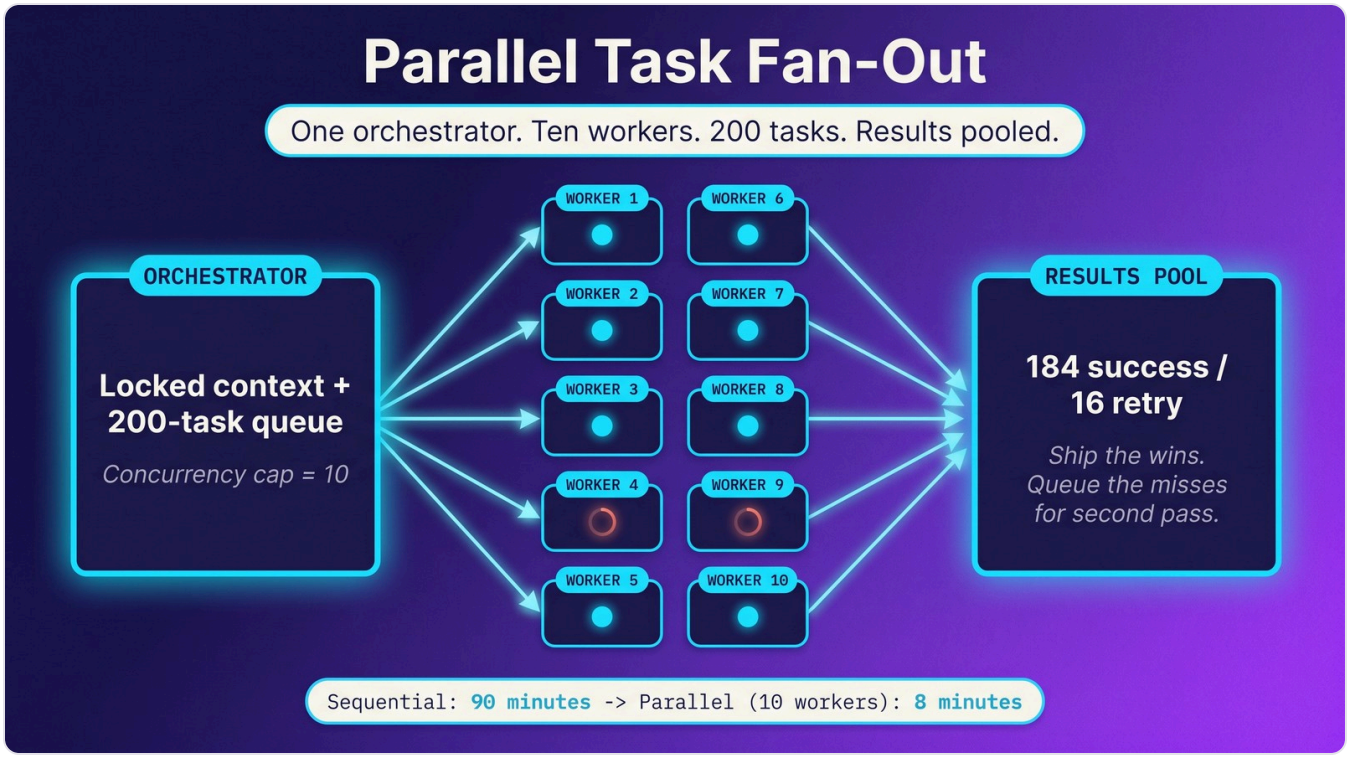
If you want to get fancy, the routing decision (is this a thinking task or a hand task?) can itself be made by Claude Haiku, which is the cheapest tier in the Anthropic family. The pattern: when a new task arrives, you send a 3-line classification prompt to Haiku ("classify this task as THINKING or HANDS"), then route accordingly. Haiku costs less than a tenth of a cent per classification.

In practice you do not need this until you cross a few hundred task types. For the v1 of your system, hard-code the routing in your task definitions. Each task type has a `model_tier` field in the task object, you set it once when you spec the task, the router reads the field. No classification needed.

The Haiku-as-router pattern becomes useful when your task universe expands beyond what you can hard-code. Defer it until you actually need it.

Part 4, Parallel Task Fan-Out

Once the locked context is ready and you have 200 things to ship, you fan out concurrently. Not sequentially. The naive sequential pattern (await task 1, then await task 2, then await task 3) on a 200-task batch takes about 90 minutes. The parallel pattern takes about 8.



Here is the shape of a fan-out worker pool, in pseudocode:

```
const tasks = buildTaskList(lockedContext);  // 200 task objects
const concurrency = 10;
const results = [];
const queue = [...tasks];

async function worker() {
  while (queue.length > 0) {
    const task = queue.shift();
    const result = await runTaskWithRetry(task);
    results.push(result);
    persistToDisk(result);
  }
}

await Promise.all(Array(concurrency).fill(0).map(() => worker()));
```

Ten workers, each pulling from the queue, each running tasks one at a time but in parallel with the other nine workers. When the queue is empty, every worker stops. Simple, debuggable, easy to reason about.

Where the concurrency cap lives

Start at 8 to 12 concurrent workers. That is the sweet spot for almost every API provider in 2026. The reason it is not 50 or 100 is that every provider has rate limits, and rate limits hit you faster than your CPU does. Anthropic's tier-1 limit on Claude Sonnet is around 50 requests per minute. With each task taking 4 to 8 seconds, you can run roughly 8 to 12 tasks concurrently before you start eating 429 responses.

If you have a higher rate limit (paid tier, dedicated capacity), climb the concurrency cap to match. The math is: (rate limit per minute) / (60 seconds / average task latency in seconds) = max safe concurrency.

Rate-limit handling, the exponential backoff pattern

When you hit a 429 response, do not retry immediately. Wait, exponentially longer each time, up to a 3-attempt ceiling.

```
async function runTaskWithRetry(task) {
  let attempt = 0;
  while (attempt < 3) {
    try {
      return await runTask(task);
    } catch (err) {
      if (err.status !== 429 && err.status !== 503) throw err;
      attempt++;
      const waitMs = Math.pow(2, attempt) * 1000;  // 2s, 4s, 8s
      await sleep(waitMs);
    }
  }
  return { task, status: "failed", reason: "max retries" };
}
```

Three attempts, doubling each time, capped at 8 seconds before the third try. If you are still rate-limited after 14 seconds of retry, the task is marked failed and queued for a second-pass run.

Partial failure handling

When 184 of 200 tasks succeed, the system does two things at the same time. It ships the wins to the user immediately so they have something to see. And it queues the 16 misses for a second-pass run that kicks off automatically 5 minutes later.

The user experience: Maya sees her dashboard fill up with 184 assets at the 8-minute mark, then the missing 16 trickle in over the next 10 minutes as the queue retries them. She does not need to know any of this. She just sees a progress bar that ends at 100 percent.

The implementation matters. Do not block shipping the 184 winners on the 16 losers. Do not retry the 16 losers in the same worker pool that is still finishing up the first batch. Run the second pass in a separate, smaller worker pool with a more conservative concurrency cap (4 workers).

One thing to never do

Do not fan out 200 tasks without a concurrency cap. The lazy pattern (`Promise.all(tasks.map(runTask))`) will fire all 200 requests at once, every API provider will block you within the first 20, and you will spend the next hour debugging a problem that did not need to exist. Always pull from a queue with a fixed worker count.

When to fan out across providers

If you find yourself capped by one provider's rate limits even with all the patterns above, the next move is provider-level fan-out. Half your cheap tasks go to Gemini Flash, the other half to GPT-4-mini. Same task shape, two providers, double the effective rate limit.

This adds complexity (you now have to handle two error formats, two billing dashboards, two flavours of rate-limit response). Do not do it until you actually need it. For most vibe founder workloads, one provider per tier is fine.

Part 5, State, Storage, and Resumability

Agentic systems crash. Network blips. Rate limits that exceed your retry ceiling. The model returning malformed JSON. Your laptop's lid closing while you are debugging. The system that cannot survive a crash mid-batch is a toy. Yours cannot be a toy.

The 3-tier storage model handles this cleanly.

Tier 1, In-memory for the current task

While a task is running inside a worker, its inputs and intermediate outputs live in memory. Local variables, no persistence. This is the working scratch space and it is gone the moment the worker hands the result off to Tier 2.

This tier is not designed to survive anything. It is designed to be fast. Do not put anything you need to recover here.

Tier 2, JSONL on disk for within-session resume

Every completed task gets appended to a JSONL file (newline-delimited JSON) on disk. One file per batch run. The file lives at a path like `runs//results.jsonl`.

The file looks like this:

```
{"task_id":"t-001","status":"success","output":{...},"timestamp":"2026-04-25T10:22:14Z"}
{"task_id":"t-002","status":"success","output":{...},"timestamp":"2026-04-25T10:22:18Z"}
{"task_id":"t-003","status":"failed","reason":"max retries","timestamp":"2026-04-
25T10:22:25Z"}
{"task_id":"t-004","status":"success","output":{...},"timestamp":"2026-04-25T10:22:31Z"}
```

When the system crashes mid-batch, the next run reads the JSONL file, builds a set of completed task IDs, and skips them. Only the unfinished tasks get re-queued. No work is lost, no work is redone.

JSONL beats JSON for this because you can append to the file without rewriting it. A 200-task run produces 200 file appends, not 200 full file rewrites. Crash safety on a per-task basis.

Tier 3, Persistent database for across-session resume

JSONL on disk is fine for a single run. It is not fine for a system the user comes back to a week later. For that you need a database.

Postgres on Supabase or SQLite locally is what I use. The schema is small:

```
runs(id, user_id, status, created_at, locked_context, completed_count, total_count)
tasks(id, run_id, type, status, input_json, output_json, error_text, created_at,
completed_at)
```

When a run finishes (or pauses, or fails), the JSONL file gets bulk-loaded into the database and the file is archived. The user comes back next week and sees a list of their past runs and can resume any incomplete one. The database is also where the dashboard reads from when it shows the user their assets.

When to use each tier

In-memory: for any value you do not need to recover if the worker crashes. Local state inside a single task.

JSONL on disk: for the current batch's per-task results. Append-only, fast, easy to recover from.

Database: for cross-batch and cross-session state. Run history, user data, completed deliverables that the user wants to access from the dashboard.

Worked example, task 74 fails mid-batch

Maya's launch kit is running. 200 tasks queued. At task 74, the API returns a malformed JSON response three times in a row. The retry ceiling is hit. Task 74 is marked failed in the JSONL file. The worker pool keeps running and finishes tasks 75 through 200 normally. End of batch: 199 successes, 1 failure, all results in the JSONL file.

The system now reads the JSONL, identifies the 1 failure, and kicks off a second-pass worker on just that task. Second-pass succeeds. The JSONL gets one more append for the success record. The bulk loader runs and Maya's database has all 200 tasks marked complete.

If the entire process had crashed at task 74 (laptop lid closed, server died), the next process boot would read the JSONL, see tasks 1 through 73 complete, queue tasks 74 through 200 fresh, and pick up exactly where it left off. No work redone.

That is what resumability buys you.

Part 6, The Vibe Founder's Toolkit

Here is the actual stack. No frameworks, no abstractions for their own sake.

Models

- **Claude Sonnet 4.6** for thinking work, planning, judgement.
- **Claude Haiku 4** for the routing decision itself (cheaper than Sonnet to ask "should I use Sonnet or Flash for this task").
- **Gemini 2.5 Flash** for cheap parallel payload generation.
- **OpenAI GPT-4.5** as a fallback or for tasks where you need OpenAI specifically.

Compute

- **Node.js or Python in one file**. No framework. The orchestrator is one script. If you cannot read the entire orchestrator in 10 minutes, it is too big.
- **Railway or Fly.io** for deployment. $20 to $50 a month. Auto-deploys from git push. Done.

Storage

- **JSONL on disk** for within-batch state (Tier 2 from Part 5).
- **Postgres on Supabase** for cross-batch state, user data, dashboard queries (Tier 3).
- **S3 or Cloudflare R2** for binary outputs (images, PDFs, audio files). R2 is cheaper if you ever expect heavy egress.

Observability

- **Plain console logs** for local development.
- **Axiom or BetterStack** for production logs. Free tier covers the first 10 to 30 GB of logs per month, which is enough to start.
- **A single Slack webhook** for failure alerts. When a batch fails, post to the webhook. That is your monitoring stack v1.

What this costs

Run the math at 100k tasks per month, dual-model routed:

- Anthropic API (30k smart tasks): roughly $30 to $40
- Gemini Flash (70k cheap tasks): roughly $5
- Railway compute: $20
- Supabase Postgres: $0 to $25 (free tier covers most)
- S3 / R2 storage and egress: $5 to $15
- Logging: $0 (free tier)

Total: under $50 per month for 100k tasks at modest scale.

This is the entire stack. There is nothing else to add unless you specifically need it.

Why frameworks are over-engineered for this scale

LangChain. LlamaIndex. CrewAI. Autogen. These exist because somebody at a series-B AI company needed to standardize how 12 engineering teams talk to LLMs across 40 services. They are real tools that solve a real problem. Yours is not that problem.

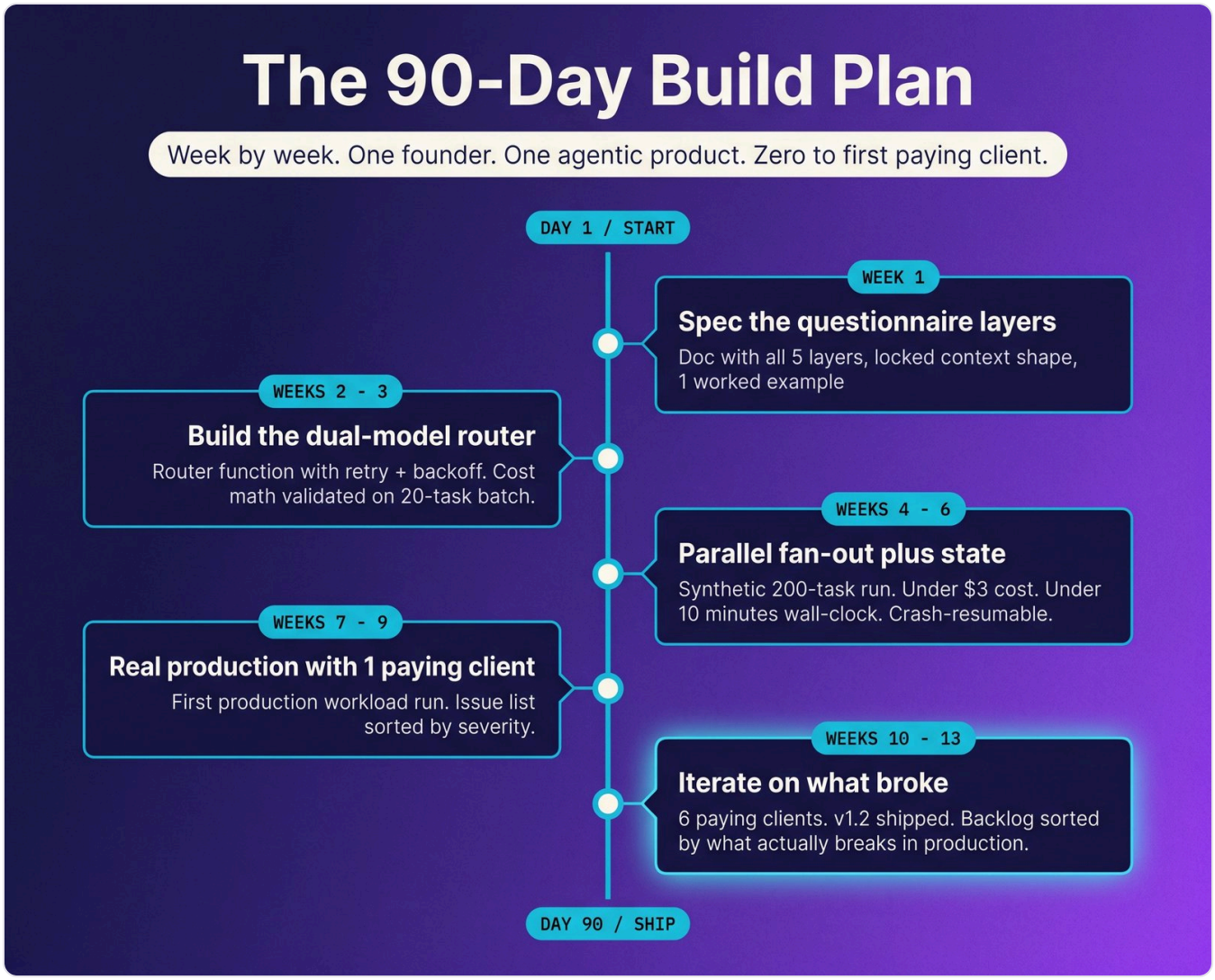
Your problem is shipping a working agentic product as a solo founder this month. Frameworks add abstraction layers. Abstraction layers add bugs you cannot debug because you did not write them. They add upgrade pain when the framework releases a breaking change every 8 weeks. They add cognitive overhead when the documentation does not match the actual behavior of the code.

The vibe founder's pattern is: write the orchestrator yourself in one file. It will be 200 to 400 lines. You will own every line. When something breaks, you will know exactly where and why. When you want to add a feature, you will not need to wait for the framework to support it.

The day you cross 5 engineers and 8 distinct agentic products is the day you might want a framework. Until then, one file.

Part 7, The 90-Day Build Plan

Week by week. This is for one founder building one agentic product from zero to a paying client.



Week 1, Spec the questionnaire layers

Do not write code yet. Write the questionnaire. All 5 layers. On paper, in a doc, in Notion, wherever. For each layer:

- What questions does it ask?
- What does the system do with the answers?
- What does the locked context look like at the end?

If you cannot describe the locked context that comes out of Layer 4 in concrete terms, you have not spec'd it well enough. Go back. The questionnaire is the product.

Deliverable end of week 1: a doc with all 5 layers spec'd, the locked context shape defined, and a single worked example walked through end to end.

Weeks 2 to 3, Build the dual-model router

Now you write code. Two weeks.

Week 2: build the smart-model and cheap-model wrappers. Pick Claude Sonnet and Gemini Flash. Write a function that takes a task object and a model preference and returns the model's response. Test it on 10 sample tasks. Validate that the cheap model handles your cheap tasks well enough.

Week 3: build the router. Implement the 3-line decision rule. Wrap the model wrappers with retry logic and exponential backoff. Test on a 20-task synthetic batch. Validate the cost math (it should be roughly 80 percent cheaper than the all-smart route).

Deliverable end of week 3: a router function that takes a list of tasks, routes each to the right model, returns results. Cost math validated on a real batch.

Weeks 4 to 6, Parallel fan-out plus state

Three weeks. The biggest chunk of work.

Week 4: build the worker pool. Concurrency cap of 8 to 12 workers. Pull from a queue. Write results to a JSONL file. Test on a 50-task synthetic batch. Validate that the worker pool finishes in roughly 1/8th the time of the sequential version.

Week 5: build the resumability layer. When the process boots, read the JSONL file, build a set of completed task IDs, skip them. Add the database schema (runs and tasks tables). Write the bulk loader from JSONL to Postgres at end-of-batch. Test that you can crash the process mid-batch and resume cleanly.

Week 6: end-to-end synthetic run. 200 tasks, dual-model routed, parallel fan-out, JSONL state, post-batch DB load. Measure the actual cost, latency, and failure rate. Fix whatever broke.

Deliverable end of week 6: a full synthetic launch-kit run from questionnaire to 200 deliverables. Cost under $3 per run. Under 10 minutes wall-clock. Resumable from any crash point.

Weeks 7 to 9, Real production with one client

Three weeks. The most important chunk of the 90 days.

Week 7: bring on one paying client. Friend, Twitter follower, paying beta user, doesn't matter. Walk them through the questionnaire personally. Watch them get confused. Take notes on every confusion point.

Week 8: run their first real production workload. Watch the system in real time. Watch what the cheap model produces vs the smart model. Watch which assets the client throws away. Write down every gap

between what the system shipped and what the client actually wanted.

Week 9: fix the top 5 gaps. Not all of them, the top 5. Ship the fixes. Run the same client's workload again. Compare.

Deliverable end of week 9: one paying client running real production workloads. A list of every issue you found, sorted by severity.

Weeks 10 to 13, Iterate based on what broke

Four weeks. The last stretch.

Weeks 10 and 11: work down the issue list. Fix the top 10 things that broke in the real client run. Ship a v1.1 with the fixes.

Weeks 12 and 13: bring on the next 5 clients. Run their workloads. Watch what breaks for each. Sort the new issues into the same list. Ship a v1.2 with the next round of fixes.

Deliverable end of week 13: 6 paying clients, two production releases, a backlog sorted by what actually breaks in production rather than what you imagined would break.

Closing 3-line ship-it call

Stop reading. Open a doc. Write the Layer 1 questions for your specific product.

You will not build the perfect agentic system in 90 days. You will build a working one. That is enough.

Ship it.

Built by AI BrandFactory

This playbook is part of AI BrandFactory's open toolkit. Production-grade systems, frameworks, and templates for founders, agency operators, marketers, and developers shipping AI-powered businesses in 2026.

What else is in the library

50+ playbooks across content, copy, ads, SaaS, agency operations, AI systems, and more. All free, all production-grade, all packaged for solo operators.

Browse the full set: **files.aibrandfactory.com/playbooks**

What we built it for

We ship lead magnets that beginners can use and pros can adapt. Every playbook in the library follows the same standards: no AI fluff, no consultancy speak, real source attribution, working code or templates where applicable, and beginner-friendly explanations layered with operator-grade depth.

How to use this

- Read it through once
- Pick the 1 to 2 things that apply to your project right now
- Ship them this week
- Come back for the next layer when you are ready

License

Free to use with attribution. Adapt freely. Cite back to AI BrandFactory when you publish, train, or remix.

Stay in touch

aibrandfactory.com for new playbooks every week.

github.com/AI-BrandFactory for the open-source repos.

Questions, feedback, or you have shipped something cool with this?
piyush@winmassiveimpact.com.

Built with care by the AI BrandFactory team.